

■ SignSpeak

Complete Project Explanation

AI-Powered Sign Language Translation System

From Beginning to End — Every Detail Explained

Technology Stack	React (Vite) · FastAPI · TensorFlow · SQLite · MediaPipe
Deployment	Vercel (Frontend) · Railway (Backend + AI Model)
Repository	github.com/ngoh-remy/signspeak2
AI Model	LSTM Neural Network · 50 Signs · 1,662 features/frame
Lifecycle	MLLC + SDLC (Merged at Deployment)
Methodology	Agile Iterative Development

1. What Is SignSpeak?

SignSpeak is a full-stack, AI-powered web application that translates sign language gestures into text and spoken audio in real time using only a standard webcam. Think of it like Google Translate, but instead of typing words, you perform hand signs in front of your camera. The application watches your movements, recognizes the sign, and instantly writes the word on screen and reads it out loud.

It is a complete, production-ready product — not just a script or demo. It includes a live website hosted on the cloud, a secure user account system, a sign dictionary, a sentence builder that accumulates multiple signs, bilingual support (English and French), dark and light mode, and persistent translation history saved to a database.

■ Live at: <https://signspeak2.vercel.app> · Backend: <https://signspeak2-production.up.railway.app>

2. The Problem We Are Solving

People who are deaf or hard of hearing communicate primarily through sign language. However, the vast majority of the general population does not understand sign language, creating a daily communication barrier. Existing solutions — such as human interpreters — are expensive, not always available, and cannot scale to millions of people.

SignSpeak proposes a technological bridge: an AI that can watch a person signing and automatically convert those gestures into words that anyone can read or hear, accessible from any modern web browser with a camera.

3. The Lifecycles We Followed

Because SignSpeak is both an AI system AND a web application, we could not follow a single lifecycle. We merged two independent lifecycles that ran in parallel and joined together at the deployment stage.

3.1 The Machine Learning Lifecycle (MLLC)

This lifecycle governs the AI brain of the application:

Phase	What We Did
1. Problem Definition	Define the goal: translate ASL gestures to text in real time.
2. Data Acquisition	Record MP4 videos for 50 signs (4 videos per sign).
3. Data Preprocessing	Run MediaPipe on every video to extract 1,662 coordinates per frame.
4. Model Training	Train an LSTM neural network on the preprocessed keypoint data.

5. Model Evaluation	Measure accuracy and loss, generate training graphs.
6. Deployment	Push model.h5 to the Railway cloud server.
7. Monitoring	Tune the confidence threshold in real-world testing.

3.2 The Software Development Lifecycle (SDLC)

This lifecycle governs the website infrastructure:

Phase	What We Did
1. Planning	Define features: auth, translate, dictionary, history, bilingual UI.
2. System Design	Design database schema, API endpoints, UI pages, and routing.
3. Implementation	Write React frontend, FastAPI backend, and SQLite database code.
4. Testing	Write unit, integration, and end-to-end tests.
5. Deployment	Deploy frontend to Vercel, backend to Railway via Docker.
6. Maintenance	Fix bugs, add dark/light mode, polish mobile responsiveness.

The Merge Point: When we embedded model.h5 from the MLLC into the FastAPI backend of the SDLC via inference.py, the two lifecycles became one unified system.

4. The Methodology: Agile Iterative Development

Our overarching management methodology is Agile. Instead of the old 'Waterfall' approach (plan everything → build everything → test at the very end), we worked in short, repeated cycles. Each cycle produced a working, testable, and deployable piece of software.

Every push to GitHub represents one completed Agile sprint. The CI/CD (Continuous Integration / Continuous Deployment) pipelines on Vercel and Railway ensured that every approved change went live automatically within minutes.

5. Project Structure Overview

Folder / File	Role
Ai_model/	The AI brain — data, preprocessing, training, and inference
Ai_model/SL/	Raw MP4 video dataset (organized by sign name)
Ai_model/keypoints/	Preprocessed .npy files (30×1,662 arrays per video)

Ai_model/preprocess.py	Converts raw videos into numerical keypoint arrays
Ai_model/train.py	Builds and trains the LSTM neural network
Ai_model/inference.py	Real-time prediction engine used by the backend
Ai_model/model.h5	The trained AI brain file (6.7 MB)
Ai_model/labels.json	Ordered list of 50 sign names the model knows
backend/	The server — handles requests, authentication, and WebSocket
backend/main.py	FastAPI app + the WebSocket real-time recognition endpoint
backend/routes.py	All HTTP REST API endpoints (auth, signs, history)
backend/auth.py	Security: bcrypt password hashing + JWT token management
backend/database.py	SQLite database connection using SQLAlchemy ORM
backend/models/models.py	Database table definitions (Users, TranslationHistory)
backend/Dockerfile	Container recipe for Railway cloud deployment
frontend/src/	The user interface — everything the user sees in the browser
frontend/src/App.jsx	Main app entry point with page routing
frontend/src/api.js	API helper that automatically attaches JWT tokens
frontend/src/index.css	Global design system (colors, fonts, dark/light mode)
frontend/src/context/AuthContext.jsx	Global state (user, theme, language)
frontend/src/pages/Translate.jsx	The core translation engine page (WebSocket)
frontend/src/pages/Dictionary.jsx	Browse all 50 supported signs
railway.toml	Railway deployment configuration

6. Part 1 — The AI Model

6.1 Phase 1: Data Collection

The raw dataset is stored in `Ai_model/SL/`. Each subdirectory represents one sign (e.g., `SL/hello/`, `SL/thank you/`). Inside each folder are 4 short MP4 video files of a person performing that sign. The 50 signs cover everyday communication: greetings, emotions, family members, medical terms, and common verbs.

■ Limitation: Professional AI models require 50–200 videos per sign. With 4 videos per sign, the model works well for clear, consistent signs but may struggle with variations in lighting, angle, or speed. This is a dataset size constraint, not a flaw in the system architecture.

6.2 Phase 2: Data Preprocessing (`preprocess.py`)

Instead of feeding raw video pixels into the neural network (which would require massive amounts of data and computation), we use Google's MediaPipe Holistic library to extract a mathematical 'skeleton' from every frame.

How MediaPipe Works

MediaPipe scans each video frame and finds the exact pixel coordinates of body and hand joints. It outputs X, Y, and Z coordinates for:

Body Part	Landmarks	Values Each	Total Numbers
Body Pose	33 joints	4 (x, y, z, visibility)	132
Face	468 points	3 (x, y, z)	1,404 (zeroed out)
Left Hand	21 joints	3 (x, y, z)	63
Right Hand	21 joints	3 (x, y, z)	63
TOTAL PER FRAME	—	—	1,662 numbers

The face landmarks are included but set to zero. This focuses the model exclusively on hand and arm movements — the parts that actually change between signs.

Standardizing Video Length

Every video must produce exactly 30 frames to be a valid input to the neural network. If a video is shorter than 30 frames, zero arrays are appended (padding). If it is longer, 30 frames are evenly sampled across the entire duration.

Output: Each video becomes a `.npy` (NumPy array) file of shape (30, 1662) — 30 frames, each with 1,662 numerical features. These are saved in the `keypoints/` folder.

6.3 Phase 3: Model Building & Training (train.py)

Why LSTM?

A sign is a movement over time — not a frozen snapshot. Standard neural networks have no memory: they treat each input independently. LSTM (Long Short-Term Memory) networks were specifically invented for sequential data. They maintain internal memory cells that carry information from earlier frames to later ones.

Example: LSTM sees Frame 1 (hand at rest), Frame 8 (hand moving up), Frame 15 (hand waving), Frame 30 (hand returning). It combines all this information to conclude: 'hello'.

Model Architecture

Layer	Type	Size	Purpose
1	LSTM	64 units	Learns low-level temporal movements (early frames)
2	BatchNormalization	—	Stabilizes and speeds up training
3	Dropout 30%	—	Randomly disables neurons to prevent memorization
4	LSTM	128 units	Learns higher-level sign patterns across all 30 frames
5	BatchNormalization	—	Stabilizes training
6	Dropout 30%	—	Further prevents overfitting
7	Dense (ReLU)	64 units	Combines learned features into a classification signal
8	Dropout 50%	—	Final regularization
9	Dense (Softmax)	50 units	Outputs a probability (0–1) for each of the 50 signs

Training Process

The 200 total samples (50 signs × 4 videos) are split: 80% for training, 20% for testing. The model trains for up to 100 epochs. Three callbacks monitor training automatically:

Callback	Purpose
ModelCheckpoint	Saves the best model whenever validation accuracy improves
EarlyStopping (patience=20)	Stops training if accuracy stalls for 20 epochs in a row
ReduceLROnPlateau	Halves the learning rate when training plateaus

Output: The best trained model is saved as model.h5 — a 6.7 MB file containing all learned weights. Training also generates accuracy.png and loss.png charts in training_results/.

6.4 Phase 4: Real-Time Inference (inference.py)

inference.py is the real-time engine. It loads model.h5 once when the server starts and then processes every camera frame sent through the WebSocket.

Step-by-step flow for each camera frame received:

1. Decode the JPEG bytes into a pixel array using OpenCV.
2. Run MediaPipe Holistic on the pixel array to extract 1,662 coordinates.
3. Add the coordinates to a rolling buffer (deque) of maximum size 30.
4. If the buffer has fewer than 30 frames, send back a 'processing' message to the frontend.
5. When the buffer reaches 30 frames, convert it to shape (1, 30, 1662) and run the LSTM model.
6. The model outputs 50 probability values. Find the highest one.
7. If the highest probability exceeds 0.05 (5%), return the corresponding sign label.
8. Clear the buffer so the next sign starts from frame 0.

■ Key settings: SEQUENCE_LENGTH=30 (frames per prediction),
CONFIDENCE_THRESHOLD=0.05 (lowered to ensure output with small dataset),
SIMULATION_MODE=False (always uses the real AI model).

7. Part 2 — The Backend Server

The backend is a Python web server built with FastAPI. It runs 24/7 on Railway's cloud infrastructure and does three things: serves the AI model's predictions via WebSocket, manages user accounts via REST API, and stores translation history in a SQLite database.

7.1 The Database

SQLite is used for the database — a file-based system that requires no separate server. SQLAlchemy (an ORM) lets us interact with the database using Python classes instead of raw SQL queries. Two tables are defined:

users table

Column	Type	Purpose
id	Integer (PK)	Auto-incrementing unique identifier
username	String (unique)	The user's display name
email	String (unique)	Login email address
hashed_password	String	bcrypt-encrypted password (plain text is never stored)
is_active	Boolean	Whether the account is enabled
created_at	DateTime	When the account was created

translation_history table

Column	Type	Purpose
id	Integer (PK)	Unique record identifier
user_id	Integer (FK)	Links to the user who performed the sign
sign_label	String	The recognized sign word (e.g., 'hello')
confidence	Float	Model confidence score (0.0 to 1.0)
session_id	String (UUID)	Groups signs from the same camera session
created_at	DateTime	When the sign was recognized

7.2 Security & Authentication

Password Hashing (bcrypt)

We never store a user's plain-text password. When you register with 'mypassword', bcrypt transforms it into a scrambled string like '\$2b\$12\$K9d7GjX...'. Even if someone steals the entire database file, they cannot reverse-engineer the original password from this hash.

JWT Tokens (JSON Web Tokens)

After successful login, the server creates a signed JWT token and sends it to the browser. The browser stores this token in localStorage and sends it with every API request in the 'Authorization: Bearer ' header. The server verifies the cryptographic signature instantly — without querying the database — making authentication fast and stateless.

7.3 The REST API Endpoints

Method	Endpoint	Purpose
GET	/api/health	Check if the server is running
POST	/api/auth/register	Create a new user account
POST	/api/auth/login	Log in and receive a JWT token
GET	/api/auth/me	Get the current logged-in user's info
POST	/api/auth/forgot-password	Request a password reset link
POST	/api/auth/reset-password	Complete password reset with token
GET	/api/signs	Get all 50 supported signs (with search)
GET	/api/history	Get the user's translation history
DELETE	/api/history	Clear the user's translation history

7.4 The WebSocket Real-Time Engine

Normal HTTP (request-response) is too slow for real-time video. WebSocket creates a persistent, two-way connection that stays open — like a phone call instead of letters. The frontend sends one JPEG frame every 100ms (10 FPS). The server processes each frame through MediaPipe + LSTM and pushes the result back the instant it's ready.

Message Type	Direction	Content
connected	Server → Browser	Confirms WebSocket is open and AI model is ready
processing	Server → Browser	Frame count update: {frames_buffered: N, frames_needed: 30}
recognition	Server → Browser	Sign result: {sign: 'hello', confidence: 0.15}
JPEG bytes	Browser → Server	Raw image bytes of the current camera frame

8. Part 3 — The Frontend (User Interface)

8.1 Technology Stack

Technology	Role
React (Vite)	JavaScript UI framework for building interactive components
React Router	Handles navigation between pages without full page reloads
Lucide React	Icon library used throughout the interface
Vanilla CSS	Custom styling — no Bootstrap or Tailwind, full design control
Web Speech API	Browser built-in API for text-to-speech audio output
WebSocket API	Browser built-in API for real-time server communication
MediaDevices API	Browser built-in API for accessing the webcam

8.2 Application Routing

URL	Page	Requires Login?
/	Landing (Home page)	No
/login	Login form	No
/register	Registration form	No
/forgot-password	Password reset request	No
/reset-password	Password reset form	No
/dictionary	Browse all 50 signs	No
/translate	Live AI translation engine	No (history requires login)

8.3 The AuthContext (Global State Manager)

AuthContext uses React's Context API to share data across all components without prop drilling. It stores the logged-in user object, the current theme (dark/light), and the active language (en/fr). It also provides `login()`, `logout()`, `register()`, `toggleTheme()`, and `changeLanguage()` functions that any component can call. All preferences are persisted in `localStorage` so they survive page refreshes.

8.4 The Translate Page — Core Engine

`Translate.jsx` is the heart of the application (477 lines). It orchestrates everything:

- Requests webcam access using `navigator.mediaDevices.getUserMedia()`
- Draws each video frame onto a hidden HTML element every 100ms

- Exports the canvas as a JPEG blob and sends it through the WebSocket
- Listens for server messages and updates the UI state (OFFLINE / CONNECTING / READY / PROCESSING)
- Displays the 'Capturing Gestures: X / 30 frames' progress bar
- Accumulates recognized signs into the Sentence Builder
- Reads each recognized word aloud using window.speechSynthesis (text-to-speech)
- Fetches and displays the user's past translations from the database
- Supports switching between front and rear cameras on mobile devices

8.5 The Design System (index.css)

All colors, fonts, and spacing are defined once as CSS custom properties (variables) in index.css. Every component uses these variables (e.g., `var(--bg-base)`) instead of hardcoded values. This is what makes the dark/light mode toggle work instantly across the entire app:

When the user clicks the Sun/Moon icon, `toggleTheme()` sets `data-theme='light'` on the root element. This triggers the `:root[data-theme='light']` CSS block, which overrides all background and text color variables. Since every component references these variables, the entire application re-themes with a single attribute change — no page reload needed.

9. Part 4 — How It All Connects

This is the most important section. Let's trace a sign from your webcam to the spoken word:

Step	Component	What Happens
1	Your Webcam	Captures a video frame every 100 milliseconds
2	Translate.jsx (React)	Draws the frame onto a hidden canvas, exports as JPEG blob
3	WebSocket connection	Raw JPEG bytes are sent to the Railway server over WSS
4	main.py (FastAPI)	Receives bytes, calls recognizer.process_frame(frame_bytes)
5	inference.py	Decodes JPEG → runs MediaPipe → extracts 1,662 coordinates
6	inference.py buffer	Coordinates added to buffer (deque maxlen=30)
7	inference.py model	When buffer=30: feeds (1,30,1662) array into model.h5 LSTM
8	model.h5 output	Returns probability array [0.15, 0.03, 0.08, ...] for 50 signs
9	inference.py threshold	Finds max probability. If >0.05: returns ('hello', 0.15)
10	main.py	Builds JSON response, saves to database (if user logged in)
11	WebSocket response	JSON sent back to browser: {type:'recognition', sign:'hello'}
12	Translate.jsx	Updates UI with the word, appends to Sentence Builder
13	Web Speech API	Browser reads 'hello' out loud using text-to-speech
14	Buffer cleared	inference.py resets buffer to 0 — cycle begins again

- Total round-trip time: approximately 3 seconds per sign (30 frames × 100ms per frame). The WebSocket stays open for the entire session — no reconnection needed between signs.

10. Part 5 — Deployment

10.1 Frontend → Vercel

Vercel is a cloud platform specialized for hosting web applications. Our React code lives in the frontend/ folder on GitHub. Vercel is connected to our repository. Every time we push to the main branch, Vercel automatically builds the React app (npm run build) and deploys the new version globally within 60 seconds. The result is available at <https://signspeak2.vercel.app>.

10.2 Backend + AI Model → Railway (Docker)

Railway runs server applications. Our Python backend and AI model are deployed using Docker. A Dockerfile is a recipe that tells Railway exactly how to build the environment:

1. Start with a clean Python 3.11 Linux environment (python:3.11-slim)
2. Install system dependencies that OpenCV and MediaPipe require (libgl1, libglib2.0-0)
3. Copy and install all Python packages from backend/requirements.txt
4. Copy the Ai_model/ folder (including the 6.7 MB model.h5 file)
5. Copy the backend/ source code
6. Start the server: uvicorn main:app --host 0.0.0.0 --port \$PORT

railway.toml configures Railway to use the Dockerfile, monitor /api/health for uptime, and restart automatically on failure. Like Vercel, Railway auto-deploys on every GitHub push.

11. Part 6 — Testing

Test Level	Tool	What It Tests	Example
Unit	pytest	Individual functions in isolation	hash_password() returns a bcrypt string; extract_keypoints() returns shape (1662,)
Integration	FastAPI TestClient	Multiple components working together	POST /api/auth/register returns 200 + JWT; GET /api/history without token returns 401
End-to-End	React Testing Library	Full user flows in the browser	User fills registration form → clicks Submit → is redirected to /translate

12. Key Technical Concepts Explained Simply

MediaPipe

A Google library that detects the exact position of hand joints, body pose, and face landmarks in a camera frame in milliseconds. We use it to convert video pixels into clean numerical coordinates, making the AI's job much simpler.

LSTM Neural Network

A type of neural network with built-in 'memory'. Unlike standard networks that treat each input independently, LSTM remembers what happened in previous frames when processing the current frame — essential for understanding movements over time.

WebSocket

A persistent two-way connection between browser and server (like a phone call, not a letter exchange). Allows the server to push results to the browser the instant they are ready, enabling real-time streaming.

JWT Token

A signed digital certificate issued after login. The browser stores it and sends it with every request. The server verifies the signature without a database lookup — fast, secure, and stateless.

bcrypt

A one-way password transformation. The original password cannot be recovered from the hash. Even if the database is stolen, user passwords remain safe.

Docker

A way to package an application with all its dependencies into a portable container. Our Dockerfile ensures the server runs identically on Railway's Linux cloud as on a local machine.

CORS

Cross-Origin Resource Sharing — a browser security rule that blocks web pages from calling APIs on different domains by default. Our backend adds special headers to explicitly allow the Vercel frontend to make requests to the Railway backend.

SQLAlchemy ORM

Object-Relational Mapper — lets us write Python classes instead of raw SQL. Safer (prevents SQL injection), more readable, and easier to maintain.

13. Accuracy & Current Limitations

Metric	Value
Number of signs supported	50
Training videos per sign	4
Total training samples	~200 sequences
AI model file size	6.7 MB
Frames per prediction window	30
Real-time latency	~3 seconds per sign
Confidence threshold	0.05 (5%)
Typical output confidence	15%–30%

- The low confidence is a dataset size issue, not an architecture issue. The system infrastructure (WebSocket pipeline, cloud deployment, user authentication, database) all work perfectly. Adding 40+ videos per sign would dramatically improve accuracy.

14. Summary for the Jury

In one sentence: SignSpeak is a real, deployed, AI-powered web application that translates sign language into text and speech in real time, built end-to-end from raw video data collection through neural network training to global cloud deployment.

Aspect	Our Approach
Lifecycles	Merged MLLC (Machine Learning) + SDLC (Software Development)
Methodology	Agile Iterative Development with CI/CD auto-deployment
AI Architecture	LSTM Neural Network (sequential gesture recognition)
Feature Extraction	Google MediaPipe Holistic (1,662 coordinates per frame)
Backend Framework	FastAPI (Python) with WebSocket real-time streaming
Frontend Framework	React (Vite) with React Router and Vanilla CSS
Security	bcrypt password hashing + JWT authentication tokens
Database	SQLite via SQLAlchemy ORM
Deployment	Vercel (frontend) + Railway/Docker (backend + AI model)
Testing	Unit + Integration + End-to-End test suites

Extras	Dark/Light mode, Bilingual (EN/FR), Mobile responsive, Text-to-speech
--------	---

- What makes this project stand out: (1) It is not a simulation — the AI is real and live. (2) Anyone can use it right now at signspeak2.vercel.app. (3) It follows professional-grade engineering practices end-to-end. (4) It demonstrates a complete, production-ready MLOps pipeline.